

The Design and Architecture of the Blacklight Library

Jackson Collins

jacksoncofficial3@gmail.com

Abstract

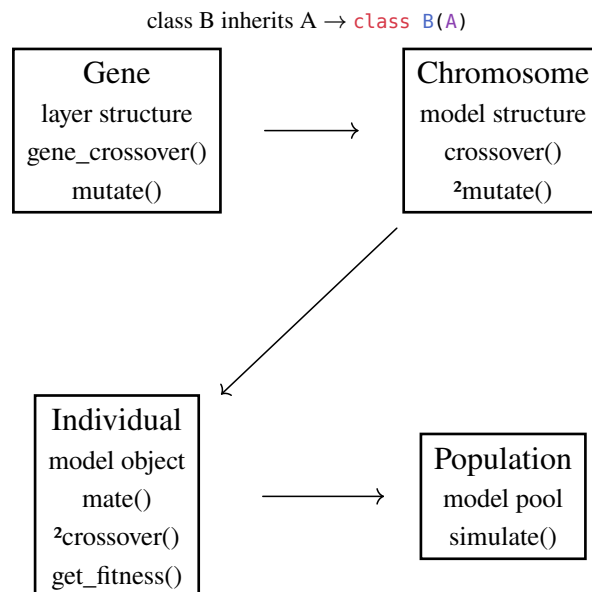
The design and architecture decisions of a project have just as much influence over the result as the code within.

Introduction

Blacklight is a python library meant to aid in the creation of topologically optimized DNNs¹ using genetic algorithms. A Blacklight model is intialized by defining model options through the ModelConfig constructor:

```
config = blacklight.ModelConfig(learning_rate=0.001, ...)
model = blacklight.create_model(config)
```

When the model is defined the ModelConfig is coded into *genes*; each gene represents one layer in the model. These genes are then collected into chromosomes, where each chromosome is a list of genes (`list[Gene]`). Chromosomes represent individual model topologies. The functionality of a chromosome is abstracted into an individual; the individual has the ability to mate, therefor it is placed into the population to interact with other topologies.



The genetic metaphor for designing a neural network allows for complex operations such as tuning layer parameters during runtime to be abstracted into simple steps that are intuitive for both the library maintainer and the user.

¹Deep Neural Networks

²While A implements the logic for some function f, B applies f to itself more generally

Inspiration

The structure and functionality of Blacklight is inspired by a paper called "Evolving Neural Networks through Augmenting Topologies," commonly NEAT. What NEAT set out to do was formulate a strong method for generating neural networks by representing the network as genes and then evolving them. NEAT is state of the art for generating neural networks with genetic algorithms but it does have a few shortcomings. The implementation of NEAT described in the 2002 paper [1] assumes the network to be made entirely out of dense/linear layers, or for the network to be a Multilayer Perceptron [2]. The other primary concern with NEAT is that of computational performance in that the model has to be constantly evaluated during evolution. While methods for dealing with the second problem are not concrete, Blacklight does intend to at least solve the first problem. By allowing the library user to specify certain attributes about the model that are then coded into *DNA*, the Blacklight library can evolve more complex networks that are able to handle more complex task than a simple MLP³.

Genes

When a user defines a models parameters they can be thought of as DNA. This DNA tells the library certain things about the model it will be creating, such as problem type, minimum/maximum number of layers (and the layer types), and the minimum/maximum number of neurons per layer. The model's DNA is computed by the library to formulate genes that define individual layers within the network. During the construction of the library it was decided that it would be best to abstract away certain functionalities of the chromosome into a gene class. This abstraction allows for certain things about the gene to be guaranteed, such as type and size.

```
class Gene:
    def __init__(self, gene_type: object=None, dna: list[Any] | None=None):
        ...
```

When `gene_type` is not passed in, a random gene is initialized according to the DNA, if there is not DNA, the gene is completely random.

Chromosomes

Chromosomes represent individual model topologies. Chromosome inherits the Gene class `Gene → Chromosome`. Since certain aspects of the gene are confirmed by their existence, the Chromosome class can use the functionality implemented in the Gene without having to worry about edge cases.

```
class Chromosome(Gene)
```

³Multilayer Perceptron

Individuals and Population

The `Individual` and `Population` classes work hand in hand for evaluating and evolving the neural network. While a human has 46 chromosomes, a Blacklight model only has 1. The `Individual` really exists to abstract some of the functionality of `Chromosome` into a way that is easier to deal with. Having the individual also helps keep up the metaphor that this project stands on top of. At the end of the the model's evolution it will represent one single individual.

Chromosome → Individual → Population

The `Population` is just as it seems, a population of `Individual` being evaluated and bred with each other. The `Population` class handles certain aspects of the Blacklight library such as running the simulation and defining global rules for the models to follow, for example you may want the individuals to mutate a bit before mating, so you pause mating for a few cycles.

Conclusion

The design and architecture of a program heavily influences how the program performs, likewise the intended functionality of a program will heavily influence its design. By representing programs in terms of metaphors for what they might be analogous to, complex functionality can be abstracted away into simple parts that make up the whole. Simulating a whole individual is complicated, but simulating each of that individual's parts is simple.

Bibliography

- [1] K. O. Stanley and R. Miikkulainen, “Evolving Neural Networks Through Augmenting Topologies,” *Evolutionary Computation*, vol. 10, no. 2, pp. 99–127, 2002, [Online]. Available: <http://nn.cs.utexas.edu/?stanley:ec02>
- [2] Wikipedia contributors, “Genetic algorithm — Wikipedia, The Free Encyclopedia.” [Online]. Available: https://en.wikipedia.org/w/index.php?title=Genetic_algorithm&oldid=1236853921
- [3] Wikipedia contributors, “Multilayer perceptron — Wikipedia, The Free Encyclopedia.” [Online]. Available: https://en.wikipedia.org/w/index.php?title=Multilayer_perceptron&oldid=1234213736
- [4] C. Agard and J. Collins, “Blacklight.” [Online]. Available: <https://github.com/BlackLightLabs/blacklight>